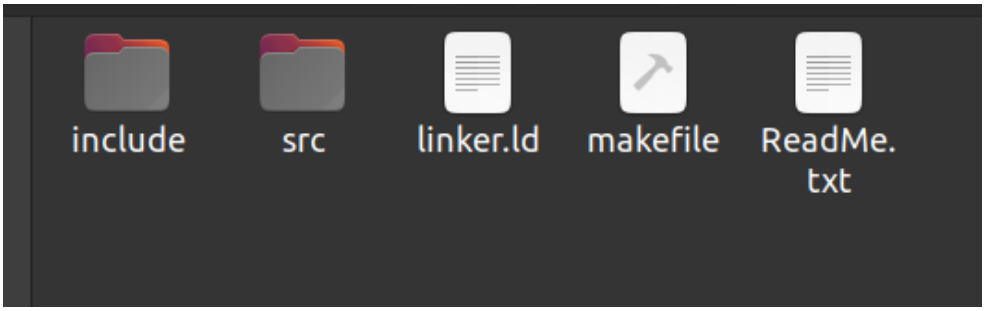
Cse312 HW1- PartB Report

R.Hilal Saygin
200104004111

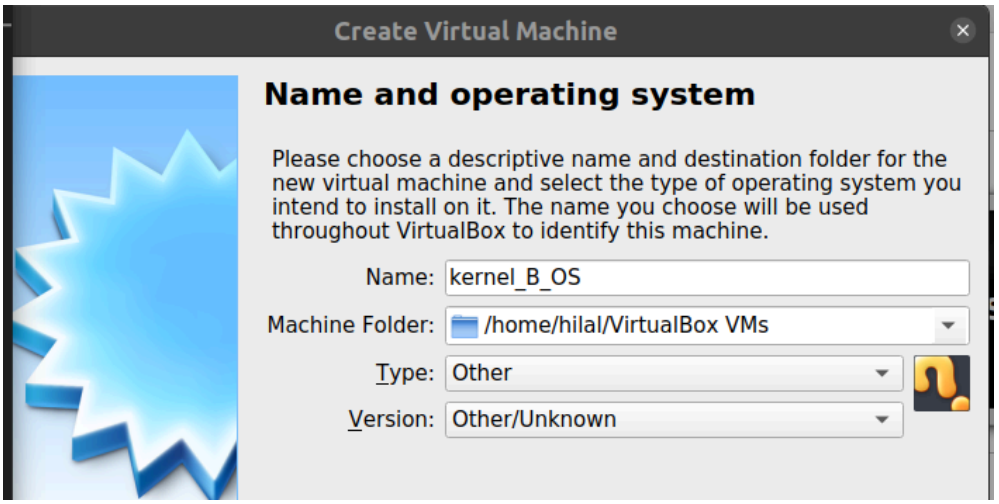
1- Install, Compilation and Run
2- General Design of kernelA.iso
System Calls:
Implemented System Calls
Process Management
Interrupt Handling
Multitasking
Context Switching
Testing Programs in Part A
3- Code Explanations
Handling Multi-Programming and Managing Process Table
Task Class:
TaskManager class:
● Attributes in TaskManager:
Scheduler:
Handling Interrupts
InterruptManager
Syscall Handler
Execution Flow
4- Test Cases and Results

1- Install, Compilation and Run

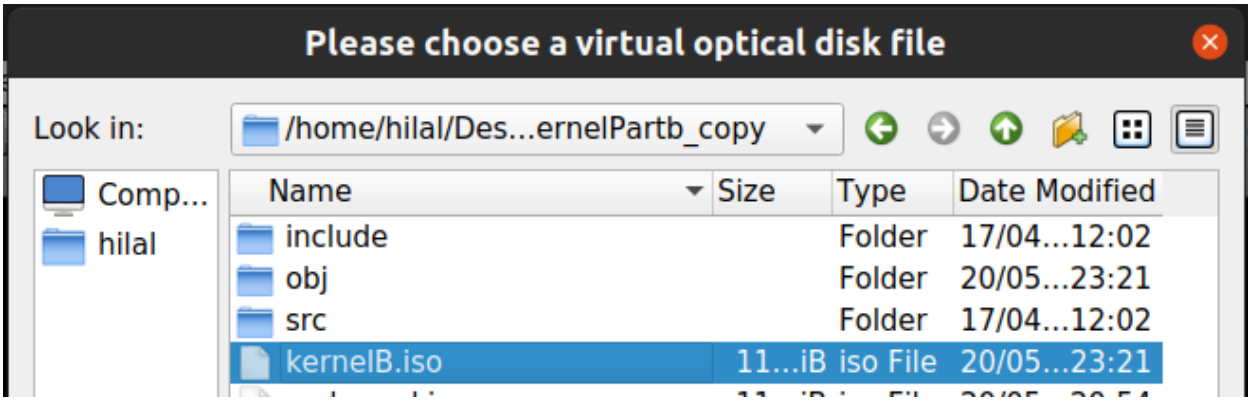
Compile the program with ‘make run’ command in the main directory where Makefile placed. When the compilation completes ISO image kernelA.iso’ will be created alongside with the .bin file.



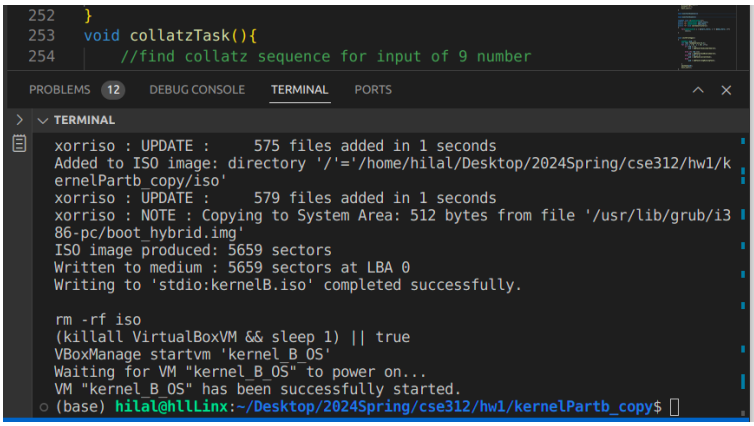
A virtual machine with the name ”`kernel_B_OS`” should be created on VirtualBox before further proceeding.

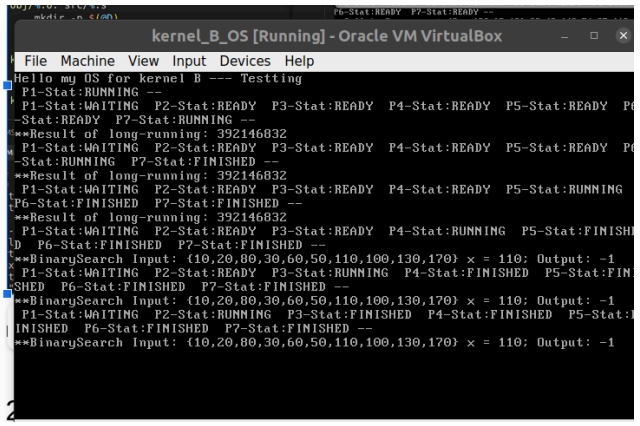


Then this created “kernelB.iso” image file should be installed as a disk file onto the machine created in the Virtualbox.
Choose the iso file directory like:



After installing the disk file to the VM in the virtualBOx, go to the main directory again and now run the ‘make’ command to start up the machine installed with kernelA.iso image file. Or You can start the machine manually from the VirtualBox Manager Interface.





If it is tested on ubuntu environment, ‘make’ will compiles, then starts vm.

2- General Design of kernelB.iso

System Calls:

The system call are same with the PartA/

Implemented System Calls

- **getPid:** Returns the process ID of the calling process.
- **waitpid:** Waits for the specified child process to change state.
- **exit_call:** Terminates the calling process.
- **sysprintf:** Outputs a string to the console.
- **fork:** Creates a copy of the calling process.
- **exec:** Replaces the process image with a new program.
- **addTask:** Adds a new task to the process table.

Testing Programs in Part B

initStrategy1

Strategy Overview :

The primary objective is to manage diverse workload by randomly selecting and adding different types of tasks to the system. This strategy tests the kernel's ability to handle different types of tasks and ensures that the task scheduler can manage and execute a variety of tasks without any issues.

Implementation Details

Random Task Selection: A random number (rnd) between 0 and 3 is generated using the implemented randGenerator function.
Based on the value of **rnd**, different types of tasks are added to the task manager. These tasks include:

TaskLinearSearch
TaskBinarySearch
collatzTask
longRunningTask

Adding Tasks:
A loop runs 10 times, and in each iteration, a task is selected randomly based on the value of rnd and added to the task manager using the addTask function.
This simulates a scenario where multiple tasks of different types are being added to the system in quick succession.

Waiting for Task Completion:

The waitpid function is called with the last added task's PID to ensure that the system waits for this task to complete before the strategy function exits.
Benefits of the Strategy

Diversity in Task Types: By randomly selecting different types of tasks, We achieve testing the kernel's ability to handle a variety of workloads.
Stress Testing: Adding multiple tasks in quick succession helps in stress testing the task manager and scheduler.
Randomization: The use of random numbers introduces variability, ensuring that the system is tested under different scenarios in each run.

initStrategy2

Strategy Overview

The primary objective of initStrategy2 is to test the kernel's ability to handle multiple instances of the same task type. This strategy focuses on verifying the consistency and reliability of the task management system when dealing with repetitive tasks.

Implementation Details

Random Task: Same with to initStrategy1, a random number (rnd) between 0 and 3 is generated.
Based on the value of rnd, one type of task is selected. However, instead of selecting different tasks in each iteration, the same type of task is selected for three consecutive additions.

Adding Multiple tasks:
A loop runs twice, and in each iteration, three instances of the same task type are added to the task manager using the addTask function. The selected task type could be:

TaskLinearSearch
TaskBinarySearch
collatzTask
longRunningTask

The waitpid function is called for each of the three added tasks (using their respective PIDs) to ensure that the system waits for these tasks to complete before the strategy function exits.
Consistency Testing: By adding multiple instances of the same task, this strategy verifies the consistency of task execution and management.
Resource Management: It helps in testing the kernel's ability to manage resources effectively when dealing with repetitive tasks.
Reliability Check: Ensures that the task manager and scheduler can handle multiple instances of the same task type without any issues.

Priority Scheduler Implementation:

Problem-Solving Approach: It ensures that tasks with higher priority are given preference over tasks with lower priority. The scheduler iterates through all tasks, identifies the READY tasks, and selects the one with the highest priority to be executed next. This approach is particularly useful in systems where certain tasks need to be prioritized over others to meet deadlines or ensure critical operations are performed timely.

3- Code Explanations

Task Class:

Priority attribute added to the task class to implement priority scheduling.

```
class Task
{
    friend class TaskManager;
    private:
        static common::uint32_t pIdCounter;
        // 4 KiB stack memory space allocated for each task object
        common::uint8_t stack[4096];
        common::uint32_t pId=0; //set process id as zero if new task object cre
        common::uint32_t pPid=0;// set parent id as zero as well
        TaskState taskState;
        common::uint32_t waitPid;
        CPUState* cpustate;
        TaskPriority priority; // Add priorit attribute

    public:
        static const int procStackSize = 4096;

        Task(GlobalDescriptorTable *gdt, void entrypoint());
        //add constructor with priority
        Task(GlobalDescriptorTable *gdt, void entrypoint(), TaskPriority pr);

        Task();
        common::uint32_t getId();
        ~Task();
};
```

A constructor overload that takes the priority attribute as a parameter.

```
//overload a new constructor for creating task with a priorty attribute
Task::Task(GlobalDescriptorTable *gdt, void entrypoint(), myos::TaskPriority prt = LOW)
{


    cpustate = (CPUState*)(stack + Task::procStackSize - sizeof(CPUState));
    cpustate -> eax = 0;
    cpustate -> ebx = 0;
    cpustate -> ecx = 0;
    cpustate -> edx = 0;

    cpustate -> esi = 0;
    cpustate -> edi = 0;
    cpustate -> ebp = 0;

    cpustate -> eip = (uint32_t)entrypoint;
    cpustate -> cs = gdt->CodeSegmentSelector();
    cpustate -> eflags = 0x202;
    //add the attributes that will be used for implementing fork
    // waitPid for multiprocessing, used storing id of the process that the current process i
    taskState = FINISHED;
    priority = prt; // set  priority
    waitPid=0;
    pPid=0;
    pId=pIdCounter++;

}
```

TaskManager class:

CPUState* TaskManager::priorityScheduler(CPUState* cpustate); funtion is implemented as the method of TaksManager class.

```
class TaskManager
{
    friend class hardwarecommunication::InterruptHandler;
    private:
        Task tasks[256];
        int totTaskCount; // total count of processes being managed by task manager
        int currentTask; // currently running process' index number
        GlobalDescriptorTable *gdt=nullptr;
        int getIndex(common::uint32_t pid);
    protected:
        //added following functions to be used while handling system calls fork,exe
        common::uint32_t AddTask(void entrypoint());
        common::uint32_t ExecTask(void entrypoint());
        common::uint32_t GetPId();
        common::uint32_t ForkTask(CPUState* cpustate);
        bool ExitCurrentTask();
        bool WaitTask(common::uint32_t pid);
    public:
        void PrintProcessTable();

        TaskManager(GlobalDescriptorTable *gdt);
        ~TaskManager();
        bool AddTask(Task *task);
        CPUState* robinScheduler(CPUState* cpustate);
        CPUState* priorityScheduler(CPUState* cpustate);
};
```

Implementation of CPUState* TaskManager::priorityScheduler(CPUState* cpustate):

Enum added to indicate the tasks priorities:

```
enum TaskPriority {HIGH, MED, LOW};

namespace hardwarecommunication{

    class InterruptHandler;
```

The priorityScheduler function is responsible for managing task scheduling in a multitasking environment using a priority-based scheduling algorithm. It selects the next task to run based on the highest priority among all tasks that are in the READY state.

```
CPUState* TaskManager::priorityScheduler(CPUState* cpustate) {
        if(totTaskCount <= 0)
            return cpustate;

        if(currentTask >= 0)
            tasks[currentTask].cpustate = cpustate;

        // Find the task with the highest priority that is ready
        int to_be_scheduled = -1;
        int highest_priority = HIGH;
        for (int i = 0; i < totTaskCount; i++)
        {

            if(tasks[i].taskState == READY && tasks[i].priority <= highest_priority) {
                to_be_scheduled = i;
                highest_priority = tasks[i].priority;
            }
        }

        if(to_be_scheduled == -1) {
            // No ready task with higher priority than the current one
            return cpustate;
        }
        else{
            currentTask = to_be_scheduled;
            PrintProcessTable();
            return tasks[currentTask].cpustate;
        }
    }
```

Flow:

1. Check if there are any tasks to schedule:

2. If there are no tasks in the task manager (totTaskCount <= 0), the function returns the current CPU state without making any changes.
3. Save the current task's state:
4. If there is a current task running (currentTask >= 0), update its CPU state with the provided cpustate.
5. Find the highest priority READY task:

6. Initialize to_be_scheduled to -1 and highest_priority to the lowest possible priority (HIGH).
7. Iterate over all tasks: If a task is in the READY state and has a priority lower than or equal to the highest_priority, update to_be_scheduled with the task's index and highest_priority with the task's priority.
8. Check if a suitable task was found:
    - 8.1. If to_be_scheduled remains -1, it means no READY task was found. Return the current CPU state.
    - 8.2. Otherwise, update currentTask to the index of the highest priority READY task, print the process table, and return the CPU state of the scheduled task.

Usage:
This function is called by the system's interrupt handler to determine which task should be executed next based on priority. It ensures that higher-priority tasks are given precedence, thereby improving the system's responsiveness to critical tasks.

Handling Interrupts

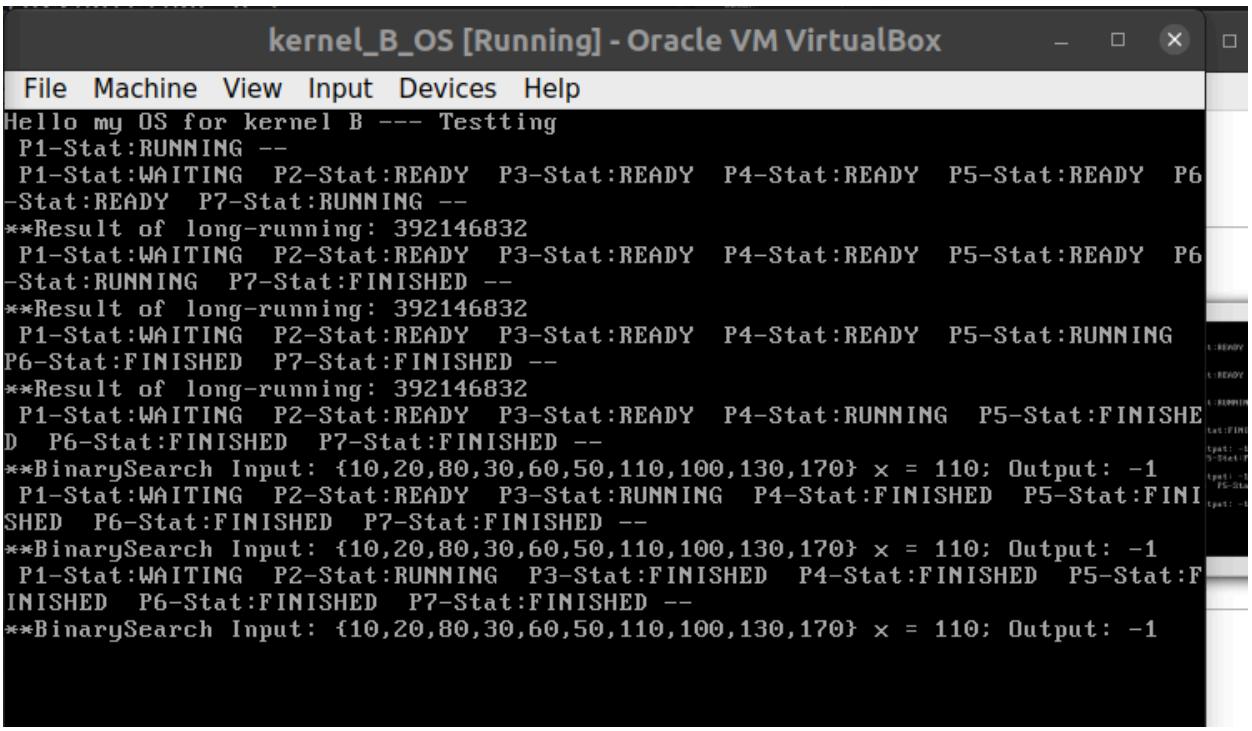
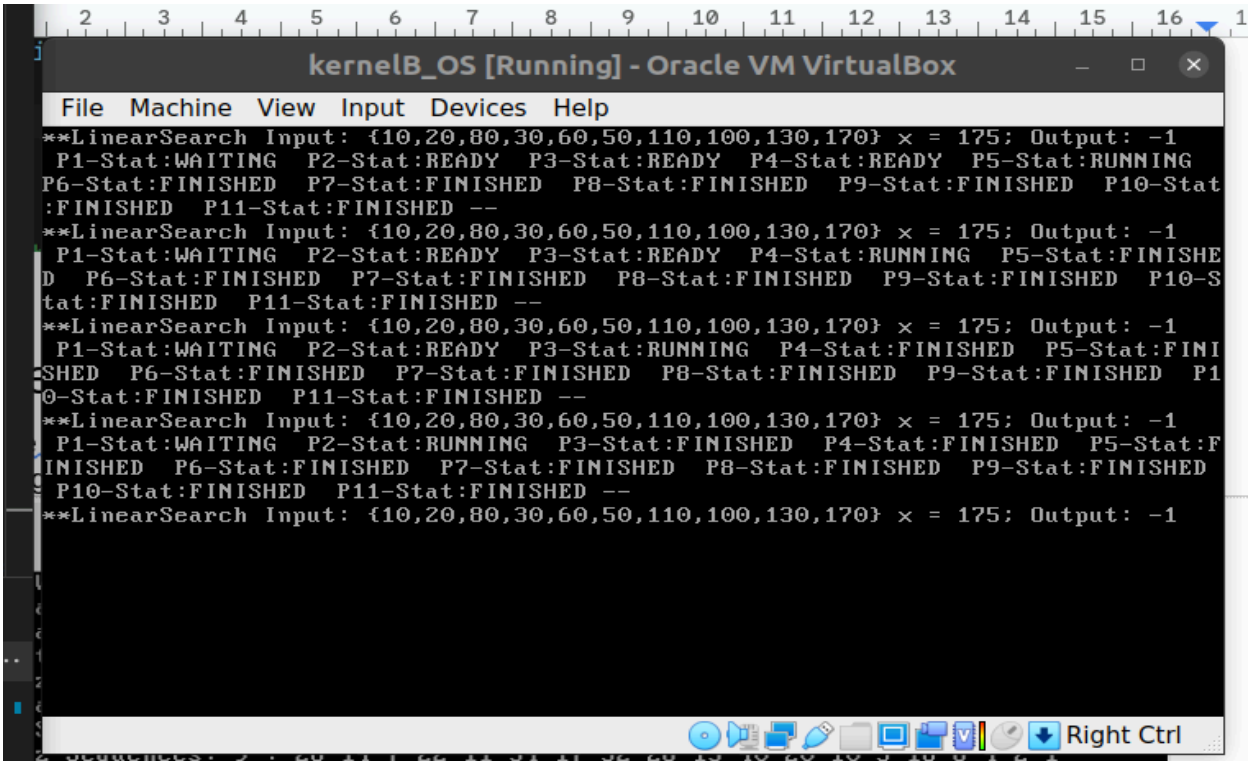
Like partA interruptHanldler is set in the kernelMain.
The priorityScheduler function of the TaskMAnager is set as the active function to be executed when interrupts occurs

```
uint32_t InterruptHandler::HandleInterrupt(uint32_t esp)
{
    // return (uint32_t)interruptManager->taskManager->robinScheduler((CPUState*)esp);
    return (uint32_t)interruptManager->taskManager->priorityScheduler((CPUState*)esp);

    // return esp;
}
```

```
if(interrupt == hardwareInterruptOffset)
{
    // esp = (uint32_t)taskManager->robinScheduler((CPUState*)esp);
    esp = (uint32_t)taskManager->priorityScheduler((CPUState*)esp);

}
```

4- Test Cases and Results

Results of initStrategy1:



Results of initStrategy2: